Loops → Batch Processing & Data Ingestion

Real-world use case

Processing large volumes of records, files, or API responses

Data engineers rarely process **one thing**:

- Loop through millions of rows from an API
- Loop through files in cloud storage
- Loop through tables for daily refresh jobs

Points

- A for loop mirrors *batch ingestion*:
  - "For each record → validate → transform → load"
- range() represents **controlled batch sizes**
- Nested loops represent **joins or multi-level data structures**

Example scenario

- Loop through incoming API records to load them into a warehouse
- Loop through a list of dates to backfill historical data

What goes wrong if loops aren't used properly

Infinite while loops

- Pipeline never finishes
- Compute costs explode
- Airflow jobs timeout

Off-by-one errors

- Misses the last record
- Duplicates data
- Partial loads with no obvious error

Excessive nesting

- Jobs slow down exponentially
- Pipelines fail at scale even if they worked locally

> **Note!**
> "A loop that works for 10 records may destroy production at 10 million."

Dictionaries → JSON, API Data & Semi-Structured Data

Real-world use case

Working with API responses, event data, and JSON payloads

Most modern data sources return **dictionaries (JSON objects)**:

- API responses
- Event logs
- Configuration files

Points

- Keys = column names
- Values = data values
- Dictionaries enable **lookup by meaning**, not position

Example scenario

- Extract "order_total" from an API response
- Store configuration values (host, port, db_name)
- Build a dictionary of metrics by name

What goes wrong if dictionaries aren't used properly

KeyErrors crashing pipelines

- API changes field name
- Missing optional fields
- One bad record crashes the whole job

Using lists instead of dictionaries

- O(n) searches instead of O(1) lookups
- Massive performance issues at scale

Poor key choices

- Using non-unique identifiers
- Overwriting records silently

> **Note!**
> "If you don't control your keys, you don't control your data."

Dictionaries vs Lists → Data Modelling Decisions

Real-world use case

Choosing the right structure for the data shape

Talking points

- Use **lists** for:
  - Ordered collections
  - Duplicate values
- Use **dictionaries** for:
  - Entity lookups
  - Configuration
  - Records with properties

Example scenario

- List of file names → list
- Customer profile → dictionary
- Customer ID → customer record → dictionary of dictionaries

What goes wrong if chosen incorrectly

Inefficient lookups

- Searching lists repeatedly
- Slow pipelines

Duplicate handling problems

- Lists silently accept duplicates
- Business metrics become inaccurate

Complex code to compensate

- Extra loops
- Fragile logic

> **Note!**
> "Data modelling mistakes show up first in performance, not syntax."

Exceptions → Reliable Production Pipelines

Real-world use case

Handling bad data, missing files, failed API calls

In production:

- Files go missing
- APIs return unexpected data
- Formats change without warning

Points

- Exceptions are **normal** in data engineering
- The goal is **controlled failure**, not avoiding failure
- try-except allows pipelines to:
    - Log the problem
    - Skip bad records
    - Continue safely

Example scenario

- Skip corrupt records but load the rest
- Retry transient API failures
- Gracefully fail with clear logs

What goes wrong if exceptions aren't handled correctly

No exception handling

- Pipeline crashes on first bad record
- Data outages

Over-using except: without specificity

- Hides serious bugs
- Pipelines "succeed" with broken data

Catching everything and doing nothing

- Silent data loss
- False confidence in dashboards

> **Note!**
> "A pipeline that never fails is usually lying."

Functions → Maintainable & Testable Pipelines

Real-world use case

Reusable logic in ETL jobs

Data engineers write the same logic repeatedly:

- Validate records
- Transform fields
- Apply business rules

Points

- Functions enforce **single responsibility**
- Functions make pipelines:
  - Easier to test
  - Easier to debug
  - Easier to reuse

Example scenario

- transform_order(record)
- validate_schema(data)
- load_to_warehouse(rows)

What goes wrong if functions aren't used properly

Monolithic scripts

- Thousands of lines long
- Impossible to debug
- One change breaks everything

Too many arguments

- Hard to understand
- Error-prone calls

Poor argument ordering

- Incorrect data passed silently
- Logical corruption, no errors raised

> **Note!**
> “Functions protect you from future you and future teammates.”

---

*args and **kwargs → Flexible Configuration & Pipelines

Real-world use case

Configurable pipelines and extensibility

Points

- *args: unknown number of inputs
- **kwargs: configuration overrides
- Common in:
  - Database connections
  - Logging configuration
  - Frameworks (Airflow, Spark)

Example scenario

- Default database config overridden per environment
- Optional pipeline parameters

What goes wrong if misused

Over-complicated function calls

- Hard to read
- Hard to maintain

Silent misconfiguration

- Misspelled keys
- Incorrect defaults applied

> **Note!**
> “Flexibility without discipline becomes chaos.”

Googling & Debugging → Core Data Engineering Skill

Real-world use case

Unblocking yourself in production

Points

- No one memorises everything
- Googling errors is a professional skill
- Understanding **error messages** is more important than code syntax

---

What goes wrong if you don't learn this

Panic when errors occur

- Freeze during incidents
- Over-escalation

Copy-pasting fixes blindly

- Short-term fix, long-term damage

> **Note!:**
> "The best engineers don't know everything they know how to find answers."